Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus: Online

Faculty: Information Systems

Module Code: ITPNA2-12

Group: Group 1

Lecturer's Name: Sonny Kabaso

Student Full Name: Kyle Rencontre

Student Number: EDUV7579091

Indicate	Yes	No
Plagiarism report attached	✓	

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature	Date
Rencontre, Kyle	20 March 2026

Lecturer's Comments:

Marks Awarded:	%

Signature	Date

Eduvos (Pty) Ltd. (formerly Pearson Institute of Higher Education) is registered with the Department of Higher Education and Training as a private higher education institution under the Higher Education Act, 101, of 1997. Registration Certificate number: 2001/HE07/008

Table of Contents

Table of Contents 2
Question 1 3
client.py 3
WMS_Server.py 4
OUTPUT 8
Question 2 9
bookings.py 9
OUTPUT 12
Question 3 13
generate_certificates.sh 13
tls_client.py 14
tls_server.py 16
OUTPUT 20
Question 4 21
client.py 21
server.py 26
OUTPUT 32
Bibliography 34

Question 1

client.py

```
""" CLIENT SIDE """

import socket


SERVER_IP = '127.0.0.1'

PORT = 5500


def start_client():

    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)


    print("=" * 40)

    print("WMS Client")

    print("=" * 40)


    try:

        print(f"Connecting to {SERVER_IP}:{PORT}...")

        client_socket.connect((SERVER_IP, PORT))

        print("Connected to server\n")


        while True:

            print(f"What item are you looking for? ('exit' to quit)")

            message = input()


            if message.strip().lower() == "exit":

                print("Disconencting from server")

                client_socket.sendall("Exit".encode('utf-8'))
```

```
            break

        if message:
            client_socket.sendall(message.encode('utf-8'))

            reply = client_socket.recv(1024).decode('utf-8')
            print(reply, end='')

    except ConnectionRefusedError:
        print(f"[ERROR] Could not connect to server at {SERVER_IP}:{PORT}")

    except Exception as e:
        print(f" Error: {e}")

    finally:
        # Close connection
        client_socket.close()
        print("[DISCONNECT] Successful\n")

if __name__ == "__main__":
    start_client()
```

WMS_Server.py

```
""" SERVER SIDE """
import socket

SERVER_IP = '127.0.0.1'
PORT = 5500
```

```
server_running = True

items = {
    "Nails": 200,
    "Screws": 321,
    "Toolkit": 59,
    "Wood Planks": 97,
    "Sheet Metal": 153,
    "Lightbulb": 76,
}

def handle_client(client_socket, client_address):
    global server_running

    try:
        print(f"Client {client_address} connected")
        while True:
            data = client_socket.recv(1024).decode('utf-8').strip()

            # in case client presses enter
            if not data:
                continue

            # Client Exit Code
            if data.lower().strip() == "exit":
                print(f" [DISCONNECT] {client_address} sent the exit code.")
                server_running = False
```

```
            break

        # search through dictionary to find item
        if data in items and items[data] > 0:
            message = f"[ITEM FOUND] {data} : {items[data]}"
        else:
            message = f"[ITEM NOT FOUND] {data} is not in stock"

        client_socket.sendall((f"{message}\n").encode('utf-8'))
        print(f"{message}\n")

    except ConnectionResetError:
        print(f"Error: Client Closed Connection Upbruptly")
    except Exception as e:
        print(f"Error: {e}")



def start_server():
    """Start TCP Server"""

    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    server_socket.bind((SERVER_IP, PORT))

    server_socket.listen()
    global server_running
```

```
# Allows for keyboard interrupt on server side every 1 second
server_socket.settimeout(1.0)

print(f'=' * 40)
print(" TCP WMS Server Started")
print("=" * 40)
print(f" Listening on {SERVER_IP}:{PORT}")

try:
    while server_running == True:

        try:
            client_socket, client_address = server_socket.accept()

            # When client connects, remove timeout
            client_socket.settimeout(None)

            handle_client(client_socket, client_address)

            if server_running == False: break

        except socket.timeout:
            continue

except KeyboardInterrupt:
    print(f"\n\n Server interrupted by keyboard input")

finally:
```

```
    if 'client_socket' in locals():
        client_socket.close()
    server_socket.close()


    print(f" Server Closed\n")


if __name__ == "__main__":
    start_server()
```

OUTPUT

```
PS C:\Users\lkren\OneDrive\Documents\ITPNA2-T12_Projects\ITPNA2-12_FormativeAssessmen
t_Online_EDUV7579091\q1> py .\WMS_Server.py
========================================
 TCP WMS Server Started
========================================
 Listening on 127.0.0.1:5500
Client ('127.0.0.1', 62946) connected
[ITEM FOUND] Nails : 200

[ITEM FOUND] Screws : 321

[ITEM NOT FOUND] Chairs is not in stock

 [DISCONNECT] ('127.0.0.1', 62946) sent the exit code.
 Server Closed

PS C:\Users\lkren\OneDrive\Documents\ITPNA2-T12_Projects\ITPNA2-12_FormativeAssessmen
t_Online_EDUV7579091\q1>
```

```
PS C:\Users\lkren\OneDrive\Documents\ITPNA2-T12_Projects\ITPNA2-12_FormativeAssessmen
t_Online_EDUV7579091\q1> py .\client.py
========================================
WMS Client
========================================
Connecting to 127.0.0.1:5500...
Connected to server

What item are you looking for? ('exit' to quit)
Nails
[ITEM FOUND] Nails : 200
What item are you looking for? ('exit' to quit)
Screws
[ITEM FOUND] Screws : 321
What item are you looking for? ('exit' to quit)
Chairs
[ITEM NOT FOUND] Chairs is not in stock
What item are you looking for? ('exit' to quit)
exit
Disconencting from server
[DISCONNECT] Successful

PS C:\Users\lkren\OneDrive\Documents\ITPNA2-T12_Projects\ITPNA2-12_FormativeAssessmen
t_Online_EDUV7579091\q1>
```

Question 2

bookings.py

```
# QUESTION 2

from operator import itemgetter

client_data = []

FIXED_PRICE = 20


def handle_input():

    print("Client Name:")

    c_name = input()


    print("\nDestinaion:")

    destination = input()


    print("\nNumber of Passengers:")

    passengers = input()


    write_ln = f"{c_name} | {destination} | {passengers}\n"


    return write_ln


def handle_file(write_ln):

    with open("client_info.txt", "a+") as file:

        file.write(write_ln)


def order_data():

    with open("client_info.txt", "r") as file:
```

```
        for line in file:
            data = line.strip().split('|')
            if data:
                client_data.append(data)

    for clients in client_data:
        total_price = clients[2]
        total_price = total_price.lstrip()
        total_price = int(total_price)

        total_price = total_price * FIXED_PRICE
        clients.append(total_price)

    client_data.sort(key=itemgetter(3), reverse=True)

    print(f"=" * 70)
    print(f"{' NAME':<20}{'DESTINATION':<20}{'PASSENGERS':<20} PRICE")
    print("=" * 70)
    for client in client_data:
        output = f" {client[0]:<20}{client[1]:<24}{client[2]:<15} ${client[3]}\n"
        print(output)


def start_booking():

    while True:
        print(f"What would you like to do?")
```

```
    print("(A) Create a booking\n(B) View Current Bookings\n(C) Exit Application\n")
    selection = input().upper()

    if selection == 'A':
        write_ln = handle_input()
        handle_file(write_ln)
        print("Client Added Successfully\n")
    elif selection == 'B':
        order_data()
    elif selection == 'C':
        break
    else:
        print("Please Enter a Valid Option\n")


if __name__ == "__main__":
    start_booking()
    print("Closing Application")
```

OUTPUT

```
t_Online_EDUV7579091\q2> py .\bookings.py
What would you like to do?
(A) Create a booking
(B) View Current Bookings
(C) Exit Application

A
Client Name:
John Peters

Destinaion:
Paris, France

Number of Passengers:
3
Client Added Successfully

What would you like to do?
(A) Create a booking
(B) View Current Bookings
(C) Exit Application

B
=========================================================================
 NAME                DESTINATION          PASSENGERS           PRICE
=========================================================================
 Susan                 Chicago                 7              $140

 Steve                 Paris                   5              $100

 John Peters           Paris, France           3              $60

 John                  Cape Town               2              $40

 Tom                   Sydney                  1              $20

What would you like to do?
(A) Create a booking
(B) View Current Bookings
(C) Exit Application

C
Closing Application
```

Question 3

generate_certificates.sh

```
#!/bin/bash

# Generate self-signed certificate for testing


# DO NOT use self-signed certificates in production!



echo "Generating SSL certificate for testing..."



openssl req -x509 -newkey rsa:4096 -nodes -keyout server-key.pem -out server-cert.pem -days 365 -subj "//CN=localhost"


echo "Certificate generated successfully!"


echo "Files created:"


echo "  - server-key.pem (private key)"


echo "  - server-cert.pem (certificate)"
```

tls_client.py

```
import socket

import ssl


SERVER_IP = 'localhost'

PORT = 8443

SERVER_CERT = 'server-cert.pem'


def create_tls_client():

    context = ssl.create_default_context(ssl.Purpose.SERVER_AUTH)


    # Prevent SSL error without disabling the security checks

    context.load_verify_locations(SERVER_CERT)


    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)


    tls_socket = context.wrap_socket(client_socket, server_hostname=SERVER_IP)


    print(f"[CONNECTING] Connecting to {SERVER_IP}:{PORT}")

    tls_socket.connect((SERVER_IP, PORT))

    print("[SUCCESS] Connected Successfully")


    return tls_socket



def start_client():

    print("=" * 60)

    print("TLS/SSL Client")
```

```
    print("=" * 60)

    try:
        tls_socket = create_tls_client()

        cipher = tls_socket.cipher()
        if cipher:
            print("[SUCCESS] TLS Connection Established:")
            print(f"\tCipher {cipher[0]}\n\tProtocol: {cipher[1]}\n\tKey Bits: {cipher[2]}")

            message = "Hello from Secure Client ^^"
            print(f"\n[SENDING...] '{message}'")
            tls_socket.sendall(message.encode('utf-8'))

            response = tls_socket.recv(4096)
            print(f"[RECEIVED] Response from Server: '{response.decode('utf-8')}'")

            tls_socket.close()
            print(f"[CLOSED] TLS Socket Closeed Successfully!")

    except Exception as e:
        print(f"[ERROR] Could not close Connection\n{e}")
        print("[TIP] Make sure TLS Server is running")

if __name__ == "__main__":
    start_client()
```

tls_server.py

```
import socket

import ssl


SERVER = '127.0.0.1'

PORT = 8443

CERT_FILE = 'server-cert.pem'

KEY_FILE = 'server-key.pem'


def create_server():

    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)


    try:

        context.load_cert_chain(certfile=CERT_FILE, keyfile=KEY_FILE)

    except:

        print("[ERROR] Certificate files not found")

        print("Run generate_certificates.sh first")

        exit(1)


    context.minimum_version = ssl.TLSVersion.TLSv1_2


    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)


    server_socket.bind((SERVER, PORT))


    server_socket.listen(5)
```

```
    server_socket.settimeout(1.0)

    return server_socket, context


def handle_client(client_socket, client_address):
    print(f"\nConnection from {client_address[0]}:{client_address[1]}")

    cipher = client_socket.cipher()
    if cipher:
        print(f"\tCipher: {cipher[0]}")
        print(f"\tTLS Version {cipher[1]}")

    try:
        data = client_socket.recv(4096)

        if data:
            message = data.decode('utf-8')
            print(f"Received Message: {message}")

            response = f"Server Received: {message}"
            client_socket.sendall(response.encode('utf-8'))
            print(f"\tSent Response")

    except Exception as e:
        print(f"[ERROR] Client Handle Error: {e}")

    finally:
```

```
        client_socket.close()
        print(f"\tClient {client_address[0]} closed")



def start_server():
    print("=" * 60)
    print(f"TLS/SSL Server")
    print("=" * 60)
    print(f"Starting TLS/SSL Server on {SERVER}:{PORT}")
    print(f"Certificate: {CERT_FILE}")
    print(f"Private Key: {KEY_FILE}")

    server, context= create_server()

    try:
        while True:
            try:
                raw_client_socket, client_address = server.accept()
                secure_client_socket = context.wrap_socket(raw_client_socket,
server_side=True)
                handle_client(secure_client_socket, client_address)
                raw_client_socket.close()

            except socket.timeout:
                continue

    except ssl.SSLError as e:
        print(f"[SECURITY ALERT] Handshake failed for {client_address[0]}")
```

```
        print(f"Clear failure message: Unauthorized or bad certificate. Details: {e}")
        #raw_client_socket.close()

    except Exception as e:
        print(f"[ERROR] SERVER ERROR: {e}")

    except KeyboardInterrupt:
        print("\n\nServer Stopped by user")

    finally:
        raw_client_socket.close()
        server.close()
        print("Server Socket Closed")


if __name__ == "__main__":
    start_server()
```

OUTPUT

```
PS C:\Users\lkren\OneDrive\Documents\ITPNA2-T12_Projects\ITPNA2-12_FormativeAssessmen
t_Online_EDUV7579091\q3> py .\tls_server.py
============================================================
TLS/SSL Server
============================================================
Starting TLS/SSL Server on 127.0.0.1:8443
Certificate: server-cert.pem
Private Key: server-key.pem

Connection from 127.0.0.1:54578
        Cipher: TLS_AES_256_GCM_SHA384
        TLS Version TLSv1.3
Received Message: Hello from Secure Client ^^
        Sent Response
        Client 127.0.0.1 closed


Server Stopped by user
Server Socket Closed
PS C:\Users\lkren\OneDrive\Documents\ITPNA2-T12_Projects\ITPNA2-12_FormativeAssessmen
t_Online_EDUV7579091\q3>
```

```
PS C:\Users\lkren\OneDrive\Documents\ITPNA2-T12_Projects\ITPNA2-12_FormativeAssessmen
t_Online_EDUV7579091\q3> py .\tls_client.py
============================================================
TLS/SSL Client
============================================================
[CONNECTING] Connecting to localhost:8443
[SUCCESS] Connected Successfully
[SUCCESS] TLS Connection Established:
        Cipher TLS_AES_256_GCM_SHA384
        Protocol: TLSv1.3
        Key Bits: 256

[SENDING...] 'Hello from Secure Client ^^'
[RECEIVED] Response from Server: 'Server Received: Hello from Secure Client ^^'
[CLOSED] TLS Socket Closeed Successfully!
PS C:\Users\lkren\OneDrive\Documents\ITPNA2-T12_Projects\ITPNA2-12_FormativeAssessmen
t_Online_EDUV7579091\q3>
```

Question 4

client.py

```
import socket

import os



SERVER_IP = '127.0.0.1'

SERVER_PORT = 5001


FILE_STORE = 'client_files'



def send_files(file_path, client_socket):

    try:

        file_path = os.path.join(FILE_STORE, file_path)

        if not os.path.exists(file_path):

            print(f"[ERROR] File not found: {file_path}")

            return


        file_name = os.path.basename(file_path)

        file_size = os.path.getsize(file_path)


        print(f"[SENDING...] {file_name} to {SERVER_IP}")

        client_socket.sendall(file_name.encode('utf-8'))


        print(f"[SENDING...] {file_size} bytes to {SERVER_IP}")

        client_socket.sendall(str(file_size).encode('utf-8'))
```

```
        bytes_sent = 0
        with open(file_path, 'rb') as file:
            while bytes_sent < file_size:
                chunk = file.read(4096)
                if not chunk:
                    break

                client_socket.sendall(chunk)
                bytes_sent += len(chunk)
                progress = (bytes_sent / file_size) * 100
                print(f"\rProgress: {progress:.1f}%", end='', flush=True)

            print(f"\rProgress: 100.0%")
            print(f" Sent {bytes_sent:,} bytes\n")
    except Exception as e:
        print(f"[ERROR] Send file: {e}")



def req_files(file_name, client_socket):
    try:
        print(f"[REQUEST] {file_name} from {SERVER_IP}")
        client_socket.sendall(file_name.encode('utf-8'))

        file_size = int(client_socket.recv(1024).decode('utf-8'))
        print(f"\tSize: {file_size:,} bytes ({file_size/1024:.2f} KB)")

        file_path = os.path.join(FILE_STORE, file_name)
```

```
bytes_received = 0

with open(file_path, 'wb') as file:
    print(f"   Receiving data...", end='', flush=True)

    while bytes_received < file_size:
        bytes_to_read = min(4096, file_size - bytes_received)
        chunk = client_socket.recv(bytes_to_read)

        if not chunk:
            print("\nNOT CHUNK")
            break

        file.write(chunk)
        bytes_received += len(chunk)
        if bytes_received % 102400 == 0:
            progress = (bytes_received / file_size) * 100
            print(f"\r   Progress: {progress:.1f}%", end='', flush=True)

print(f"\r   Progress: 100.0%")

if bytes_received == file_size:
    print(f"[SAVED] {file_path}")
    print(f"[COMPLETE] {bytes_received:,} bytes\n")
else:
    print(f"[INCOMPLETE] File Transfer Incomplete {bytes_received}/{file_size}")
```

```
    except Exception as e:
        print(f"[ERROR] Cannot Receive File\n{e}")




def start_client():
    #Start client side
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    print("=" * 60)
    print("Multi-Client File Transfer Server Started")
    print("=" * 60)

    try:
        print(f"Connecting to {SERVER_IP}:{SERVER_PORT}")
        client_socket.connect((SERVER_IP, SERVER_PORT))
        print("Connected to Server\n")

        print(f"Please Enter your Name:\n")
        name = input()
        client_socket.sendall(name.encode('utf-8'))

        while True:
            print("Would you like to send or request a file?")
            print(" (1) SEND FILE\n (2) REQUEST FILE\n (3) EXIT PROGRAM\n")
            selection = input()
```

```
            client_socket.sendall(selection.encode('utf-8'))

            if selection.strip() == '1':
                print("Please enter file path of the file you want to send:")
                file_path = input()
                send_files(file_path, client_socket)
            elif selection.strip() == '2':
                print("Please enter requested file name:")
                file_name = input()
                req_files(file_name, client_socket)
            elif selection.strip() == '3':
                print("[EXIT] Exit Condition Met\nClosing Client Connection")
                break



    except ConnectionRefusedError:
        print(f"[ERROR] Could not connect to server at {SERVER_IP}:{SERVER_PORT}")

    except Exception as e:
        print(f" Error: {e}")

    finally:
        # Close connection
        client_socket.close()
        print("[DISCONNECT] Successful\n")

if __name__ == "__main__":
    start_client()
```

server.py

```
import socket

import threading

import os


SERVER_IP = '127.0.0.1'

SERVER_PORT = 5001

FILE_STORE = "server_files"


connected_clients = []

client_names = {}


if not os.path.exists(FILE_STORE):

    os.makedirs(FILE_STORE)

    print(f"[CREATED] Directory: {FILE_STORE}")


def handle_client(client_socket, client_address):

    # handles connecting clients

    print(f"[NEW CONNECTION] {client_address} connected")


    try:

        client_name = client_socket.recv(1024).decode('utf-8').strip()

        client_names[client_socket] = client_name


        while True:

            selection = client_socket.recv(1024).decode('utf-8').strip()

            if selection.strip() == '1':

                print(f"[RECIEVE] {client_name} is sending a file\n")
```

```
            file_name = client_socket.recv(1024).decode('utf-8')
            rec_files(file_name, client_socket, client_address, client_name)
        elif selection.strip() == '2':
            print(f"[SEND] {client_name} wants to receive a file\n")
            file_path = client_socket.recv(1024).decode('utf-8')
            send_files(file_path, client_socket, client_name)
        elif selection.strip() == '3':
            print(f"[CLIENT DISCONNECT] {client_name} ({client_address[0]}) disconnected
from the server")
            break
        else:
            print("Invalid Option")
            break

    except Exception as e:
        print(f"[ERROR] {e}")

def send_files(file_path, client_socket, client_name):
    try:
        file_path = os.path.join(FILE_STORE, file_path)

        if not os.path.exists(file_path):
            print(f"[ERROR] File not found: {file_path}")
            return

        file_name = os.path.basename(file_path)
        file_size = os.path.getsize(file_path)
```

```
        #print(f"[SENDING...] {file_name} to {SERVER_IP}")
        #client_socket.sendall(file_name.encode('utf-8'))

        print(f"[SENDING...] {file_size} bytes to {client_name}")
        client_socket.sendall(str(file_size).encode('utf-8'))

        bytes_sent = 0
        with open(file_path, 'rb') as file:
            while bytes_sent < file_size:
                chunk = file.read(4096)
                if not chunk:
                    break

                client_socket.sendall(chunk)
                bytes_sent += len(chunk)
                progress = (bytes_sent / file_size) * 100
                print(f"\rProgress: {progress:.1f}%", end='', flush=True)

            print(f"\rProgress: 100.0%")
            print(f" Sent {bytes_sent:,} bytes\n")

    except Exception as e:
        print(f"[ERROR] Could not send file\n{e}")

def rec_files(file_name, client_socket, client_address, client_name):
    try:

        print(f"[RECEIVING...] {file_name} from {client_name}")
```

```
print(f"\t({client_address[0]}:{client_address[1]})")

file_size = int(client_socket.recv(1024).decode('utf-8'))
print(f"\tSize: {file_size:,} bytes ({file_size/1024:.2f} KB)")

file_path = os.path.join(FILE_STORE, file_name)

bytes_received = 0

with open(file_path, 'wb') as file:
    print(f"   Receiving data...", end='', flush=True)

    while bytes_received < file_size:
        bytes_to_read = min(4096, file_size - bytes_received)
        chunk = client_socket.recv(bytes_to_read)

        if not chunk:
            print("\nNOT CHUNK")
            break

        file.write(chunk)
        bytes_received += len(chunk)
        if bytes_received % 102400 == 0:
            progress = (bytes_received / file_size) * 100
            print(f"\r   Progress: {progress:.1f}%", end='', flush=True)

print(f"\r   Progress: 100.0%")
```

```
        if bytes_received == file_size:
            print(f"[SAVED] {file_path}")
            print(f"[COMPLETE] {bytes_received:,} bytes\n")
        else:
            print(f"[INCOMPLETE] File Transfer Incomplete {bytes_received}/{file_size}")
    except Exception as e:
        print(f"[ERROR] Files not Recieved\n{e}")




def start_server():
    # Handles server
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server_socket.bind((SERVER_IP, SERVER_PORT))
    server_socket.listen(5)

    server_socket.settimeout(1.0)

    print("=" * 60)
    print("Multi-Client File Transfer Server Started")
    print("=" * 60)
    print(f"List6ting on {SERVER_IP}:{SERVER_PORT}")
    print(f"Waiting for Client Connections...\n")

    try:
        while True:
            try:
```

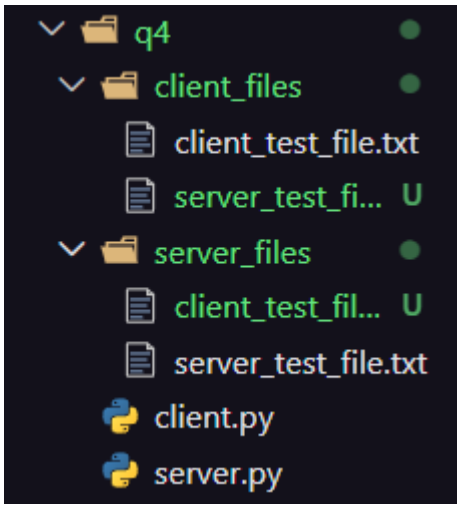
```
            client_socket, client_address = server_socket.accept()
            connected_clients.append(client_socket)

            client_thread = threading.Thread(
                target=handle_client,
                args=(client_socket, client_address)
            )
            client_thread.daemon = True
            client_thread.start()

            # Show Active Connections
            print(f"[ACTIVE CONNECTIONS] {len(connected_clients)} client(s) connected\n")
        except socket.timeout:
            continue
    except KeyboardInterrupt:
        print("\n\n Server Shutting Down")
    finally:
        for clients in connected_clients:
            clients.close()
        server_socket.close()
        print(" Server Stopped")


if __name__ == "__main__":
    start_server()
```

OUTPUT

```
PS C:\Users\lkren\OneDrive\Documents\ITPNA2-T12_Projects\ITPNA2-12_FormativeAssessmen
t_Online_EDUV7579091\q4> py .\server.py
============================================================
Multi-Client File Transfer Server Started
============================================================
List6ting on 127.0.0.1:5001
Waiting for Client Connections...

[NEW CONNECTION] ('127.0.0.1', 53737) connected
[ACTIVE CONNECTIONS] 1 client(s) connected

[RECIEVE] Kyle is sending a file

[RECEIVING...] client_test_file.txt from Kyle
        (127.0.0.1:53737)
        Size: 31 bytes (0.03 KB)
   Progress: 100.0%.
[SAVED] server_files\client_test_file.txt
[COMPLETE] 31 bytes

[SEND] Kyle wants to receive a file

[SENDING...] 31 bytes to Kyle
Progress: 100.0%
 Sent 31 bytes

[CLIENT DISCONNECT] Kyle (127.0.0.1) disconnected from the server


 Server Shutting Down
 Server Stopped
PS C:\Users\lkren\OneDrive\Documents\ITPNA2-T12_Projects\ITPNA2-12_FormativeAssessmen
t_Online_EDUV7579091\q4>
```

```
t_Online_EDUV7579091\q4> py .\client.py
============================================================
Multi-Client File Transfer Server Started
============================================================
Connecting to 127.0.0.1:5001
Connected to Server

Please Enter your Name:

Kyle
Would you like to send or request a file?
 (1) SEND FILE
 (2) REQUEST FILE
 (3) EXIT PROGRAM

1
Please enter file path of the file you want to send:
client_test_file.txt
[SENDING...] client_test_file.txt to 127.0.0.1
[SENDING...] 31 bytes to 127.0.0.1
Progress: 100.0%
 Sent 31 bytes

Would you like to send or request a file?
 (1) SEND FILE
 (2) REQUEST FILE
 (3) EXIT PROGRAM
```

```
2
Please enter requested file name:
server_test_file.txt
[REQUEST] server_test_file.txt from 127.0.0.1
        Size: 31 bytes (0.03 KB)
   Progress: 100.0%.
[SAVED] client_files\server_test_file.txt
[COMPLETE] 31 bytes

Would you like to send or request a file?
 (1) SEND FILE
 (2) REQUEST FILE
 (3) EXIT PROGRAM

3
[EXIT] Exit Condition Met
Closing Client Connection
[DISCONNECT] Successful

PS C:\Users\lkren\OneDrive\Documents\ITPNA2-T12_Projects\ITPNA
t_Online_EDUV7579091\q4>
```



Bibliography

Fadheli, A., 2025. *send-receive-files-using-sockets-python*. [Online]
Available at: https://thepythoncode.com/article/send-receive-files-using-sockets-python
[Accessed 11 March 2026].

Python Software Foundation, 2026. *socket*. [Online]
Available at: https://docs.python.org/3/library/socket.html
[Accessed 10 March 2026].

Python Software Foundation, 2026. *ssl*. [Online]
Available at: https://docs.python.org/3/library/ssl.html
[Accessed 15 March 2026].